

SRW3 Phase 1I-R1 — Formal Commitment/Gate Binding Repair Report

Branch phase1i-r1-formal-binding (from phase1i @ e04fbb6147157fb11537de68618a001fd4b9afbe, tag phase1i-complete) · **Scope** narrow formal repair of the Phase 1I K protocol model · **Not** a new research phase; no consensus-integration claim of any kind.

1. Executive summary

Phase 1I defined the protocol commitment predicate `Commit_P` and proved a twelve-claim ladder about it — but the operational transition (`pAccept`) that extends the committed chain never consulted `Commit_P`. It accepted any block whose parent link, slot and a **caller-supplied Boolean** (`SRW3OK == true`) matched, and it received a **caller-chosen configuration** at every accept. The theorem ladder pinned the meaning of the verdict outside the transition (PI7-STEP carried `Commit_P == true` as a `requires-clause`), so the theorems described a transition that never occurs as stated.

Phase 1I-R1 repairs the transition additively: the verdict argument is removed from the command surface; a machine-created **gate receipt** binds the frozen gate's verdict to the exact candidate, execution results, head and slot; the **decision is derived by the transition** from that verdict (only "valid-g" is protocol-valid, fail-closed); the **protocol configuration is pinned at initialization** and is immutable thereafter; and the guard now checks every conjunct of `Commit_P` directly. The headline theorem

R1-ACCEPT-IMPLIES-COMMITP — the operational accept guard, verbatim,

implies `Commit_P(B, ERC, ERP, C, pbParentRootOf(B)) == true` with `C` the

pinned configuration and `Commit_P` appearing **only in the destination**

is **PROVED BY K** (Haskell backend, 31 claims PROVED in total), together with non-vacuity (the honest path still commits), the forged-verdict / wrong-candidate / stale-head / wrong-slot / config-substitution rejection family, receipt single-use, configuration immutability, and all ten ported frozen claims re-run against the repaired model. The decisive test of the assignment — an invalid block remains uncommittable even when a caller attempts to supply an accepting verdict; a valid block must still be able to commit — passes mechanically (K) and concretely with real keccak-256 digests (Python, 48/48).

Scientific stopping-rule verdict: PARTIALLY CLOSED. The central binding question is closed (the repaired operational rule is proved to enforce `Commit_P` — CLOSED's criterion is met and exceeded); the verdict is PARTIALLY CLOSED because (a) the forall-closure R1-ALL remains NOT MECHANIZED (the same kore circularity implication-check blocker as the frozen PI7-ALL, retried and logged), and (b) soundness rests on two named trust boundaries stated in §8: the gate-verdict provenance (the frozen Phase 1G gate module is the only producer of "valid-g") and keccak collision resistance for the digest-deciding layer, which the Haskell backend cannot evaluate (finding 1C-1).

2. Starting point and ground rules

The repair starts from the exact base commit e04fbb6 (tag phase1i-complete) on the new branch phase1i-r1-formal-binding. The frozen branches (phase1g, phase1h, phase1h-r1-fix, phase1i) are untouched — no amend, no force-push, no file outside phase1i-r1/ (plus worklog metadata) is modified. All Phase 1I definitions — `Commit_P`, `Valid_P`, `ExecValidP`, `SRW3ValidP`, `PolicyCommitmentI`,

ProtoAuthorizedSecCtx, CtxDigestI, ProtoDecisionOfGate, the block/canonical encodings — are **inherited unchanged** by import; the repair adds commands, rules and claims only.

Toolchain (re-provisioned from the pinned recipes after the sandbox reset, all versions verified live): K framework **v7.1.337** (jammy .deb, extracted without root; kompil -version matches the Phase 1I provenance record exactly, build date Thu Jun 18 12:59:56 UTC 2026), z3 **4.13.3** (official x64 glibc-2.35 binary — the trixie-packaged z3 segfaults in this environment, as disclosed in Phase 1I), flex 2.6.4 + libfl2, libsecp256k1 0.5.0 with soname shims, Python 3.12.14 (venv) with eth-utils and pycryptodome. The LLVM backend toolchain was **not** re-provisioned; the LLVM-tier digest demonstration is disclosed as NOT RUN (same disclosure as Phase 1I) and covered by the Python keccak mirror. KEVM is not used (the repair does not touch the EVM binding).

3. The audit (what was wrong, mechanically witnessed)

Full detail in phase1i-r1/audit/FORMAL-BINDING-AUDIT.md; the four findings:

- **F1 — Commit_P is never consulted by the transition.** The frozen pAccept guard is exactly `parent-link ∧ slot-link ∧ SRW30K ==K true`.
- **F2 — the security verdict is a freely caller-supplied Bool.** A block whose decision field is pdecReject (the gate said invalid-policy) is operationally committed by pAccept(B, ERC, ERP, C, true). Witnessed two ways: the K claim OLD-ACCEPTS-INVALID-DECISION over the **unmodified** frozen module (proved by the frozen rule itself — the transition needs no validity premise of any kind), and the kron transcript T1-BEFORE-frozen-forged-verdict-ACCEPTS.txt (chain extends, counter advances).
- **F3 — PI7-STEP conditions the theorem, not the transition.** The claim adds `Commit_P == true` as a requires — proving a conditional step about a transition that, as defined, needs only the caller Boolean. This is the "premise represented as a result" pattern the handoff forbids.
- **F4 — no persistent protocol configuration; pAccept receives a caller-chosen C.** pInit(C) sets only the head; every later accept commits under whatever config the caller names. Witnessed by OLD-ACCEPTS-FORGED (the caller-chosen C2 flows into the head commitment with nothing to compare against) and concretely by the Python CM4 before-side.

4. The repair design

Additive module phase1i-r1/semantics/srw3proto-r1.k (requires the frozen srw3proto.k). Three design decisions, each mapped to the handoff's "Required design":

(a) Gate receipt — the only in-model carrier of a verdict (handoff design A "the transition computes the decision", implemented with the B "unforgeable in-model receipt" mechanism). The gate-evaluation transition pGateEval(B, ERC, ERP, V) is the **only** rule that creates a receipt; the receipt pool starts empty; an accept **consumes** its matching receipt exactly once. The receipt binds the exact candidate block, the client-derived execution results, the chain head and next-slot counter **at gate time** (freshness — a receipt minted under a displaced head cannot authorize a commit: the CM8 reorg exclusion), and the verdict string. The decision is **derived by the transition**: `grDecisionOf(R) = ProtoDecisionOfGate( grVerdictOf(R))` — the frozen total mapping under which only "valid-g" is protocol-valid and every other verdict maps to pdecReject (missing evaluation = no receipt = fail-closed).

(b) The accept command loses the verdict and the configuration. pAcceptR1(B, ERC, ERP, CV) — no SRW30K, no ProtoCfg argument. The guard enforces, directly: the receipt match (block, ERC, ERP, head, slot, derived decision valid), the decision binding (`pbDecisionOf(B) ==Int pdecValid`), execution validity (ExecValidP), the policy commitment equality **against the pinned configuration**, the authorized-context digest equality (derived from the pinned configuration and the candidate's

parent root — the applicable lineage head, the 1I convention), the parent-commitment and slot links, and the declared config-version witness CV against the pinned policy version. On accept the receipt is consumed and the chain extends; every other path falls to the [owise] reject with **state unchanged**.

(c) Configuration pinned at initialization, immutable thereafter. `pInitR1(C)` writes the pin and the genesis head (`ProtoRoot(C)`) from one argument, and **refuses to fire when a configuration is already pinned** (no in-model re-initialization). There is deliberately **no** upgrade transition: an in-model upgrade without its own authorization semantics would reintroduce the substitution attack; policy upgrade is deferred as an explicit future protocol-state transition (handoff design B, final paragraph). R1-CONFIG-IMMUTABLE proves a re-pinning attempt changes nothing.

Architecture note (why the state lives in a reserved metadata region). K allows exactly one `<k>` cell per definition, and the frozen module already declares the `<srw3proto>` configuration; K v7.1.337 does not merge configuration declarations across modules (verified by probes retained in the transcripts). The handoff permits "a persistent configuration cell (or an equivalent immutable protocol-root object)": the equivalent immutable object used here is a **reserved metadata region of the lineage map** — key -1 holds the pinned `ProtoCfg`, key -2 holds the receipt pool; block slots are always ≥ 0 , so the negative keys can never collide with a block slot, and the frozen `blockAt/CommitChainLinked` recursions match only `ProtoBlock`-valued entries (the ported PI7-BASE and the chain claims confirm). The frozen `pInit/pAccept` rules remain in the definition but are inert for repaired programs (the repaired command surface is `{pInitR1, pGateEval, pAcceptR1}`); their forged-verdict behavior is witnessed on the pure frozen definition by `old_witness.k`.

Proof-engineering adaptations (hs backend, finding 1C-1 discipline). (i) Receipt-pool claims are stated with singleton/empty concrete pool shapes: symbolic `Set` rests admit alternative AC decompositions whose guard obligations reduce to undischargable `#Ceil(Keccak256raw)` conditions (the general pool case is covered by the model's matching semantics and the Python mirror; disclosed, not hidden). (ii) The gate rule's right-hand side is a map cons-term (symbolic map-update terms do not reduce on this backend). (iii) The init-chained F4 witness OLD-ACCEPTS-CALLER-CONFIG is NOT MECHANIZED: chaining the accept after `pInit` forces the block's parent-commitment field to unify against the evaluated protocol root, leaking nested uninterpreted keccak into the block constructor; the `#Ceil` obligations cannot be discharged without evaluators — the attempt, log and blocker are retained (`transcripts/k/ kprove_OLD-ACCEPTS-CALLER-CONFIG-attempt.log`, the commented claim in `old_witness.k`).

5. The K proof ladder (31 PROVED / 1 NOT MECHANIZED)

`phases1-r1/proofs/r1_proofs.k` (spec module R1-PROOFS) + `phases1-r1/proofs/old_witness.k` (spec module OLD-WITNESS, frozen definition). Runner: `phases1-r1/proofs/run_r1_proofs.sh`; per-claim logs in `transcripts/k/`. Haskell backend, `kprove`, 300 s per-claim timeout, all run live in this environment.

#	Claim	Class	Statement (abbreviated)
1	R1-INIT-PINS	PROVED	<code>pInitR1</code> writes the pin and the protocol-root head from one argument; refuses when pinned
2	R1-CONFIG-IMMUTABLE	PROVED	with a configuration pinned, <code>pInitR1</code> changes nothing
3	R1-ACCEPT-IMPLIES-COMMITP	PROVE D	the operational guard (verbatim) $\Rightarrow \text{Commit_P}(B, \text{ERC}, \text{ERP}, C, \text{pbParentRootOf}(B))$ with the pinned C; no <code>Commit_P</code> premise
4	R1-DECISION-BOUND	PROVED	accepted candidate's decision field equals the gate-derived decision
5	R1-FORGED-VERDICT-REJECT	PROVED	recorded "invalid-policy" verdict, matched receipt, correct links $\Rightarrow$ no accept, state unchanged (CM1)

#	Claim	Class	Statement (abbreviated)
6	R1-FORGED-VERDICT-REJECT-AUTHTYPE	PROVED	same for "invalid-authtype" (representative authority branch)
7	R1-REJECT-NO-P00L	PROVED	no receipt pool $\Rightarrow$ no accept, state unchanged (CM7)
8	R1-REJECT-EMPTY-P00L	PROVED	empty pool $\Rightarrow$ no accept, state unchanged (CM7)
9	R1-REJECT-WRONG-CANDIDATE	PROVED	receipt for a different execution result $\Rightarrow$ no accept (CM5 shape)
10	R1-REJECT-STALE-HEAD	PROVED	receipt minted under a displaced head $\Rightarrow$ no accept (CM8)
11	R1-REJECT-WRONG-SLOT	PROVED	receipt from a different slot position $\Rightarrow$ no accept
12	R1-CONFIG-FRAME-GATE	PROVED	gate evaluation never mutates the pinned configuration
13	R1-CONFIG-FRAME-ACCEPT	PROVED	accept never mutates the pinned configuration
14	R1-CONFIG-SUB-REJECT	PROVED	declared version 2 against pinned version 1 $\Rightarrow$ no accept (CM4, decidable witness)
15	R1-VALID-ACCEPT	PROVED	under the full guard the transition FIRES and produces the extended state (non-vacuity)
16	R1-PIPELINE-HONEST	PROVED	init $\rightarrow$ gate("valid-g") $\rightarrow$ accept commits; receipt consumed; counter 1 (CM6, K side)
17	R1-RECEIPT-SINGLE-USE	PROVED	one receipt, two accepts: commit once, replay rejected, counter +1
18	R1-PIPELINE-FORGED	PROVED	init $\rightarrow$ gate("invalid-policy") $\rightarrow$ accept: chain unchanged, counter 0, receipt remains (CM1 end-to-end)
19	PI1-COMMIT-SAFETY, PI2-DECISION-REJECT, PI2-DECISION-UNKNOWN, PI2-EXEC-INVALID, PIB-GATELINK-VALID/REJECT1/2/3, PI5-AUTHORITY-DESCENT, PI7-BASE	PROVED	the frozen predicate-layer claims, ported verbatim and re-run against the repaired model (the predicates are unchanged definitions)
20	R1-ALL	NOT MECHANIZED	the retried forall-closure; kore aborts with the same message as the frozen PI7-ALL — "The configuration's term unifies with the destination's term, but the implication check between the conditions has failed" — log retained at transcripts/k/kprove_R1-ALL.log
30	OLD-ACCEPTS-FORGED	WITNESSED (proved)	over the unmodified frozen model: arbitrary block + verdict true $\Rightarrow$ chain extends — no validity premise of any kind
31	OLD-ACCEPTS-INVALID-DECISION	WITNESSED (proved)	the frozen rule commits a gate-REJECTED block under the caller's true
32	OLD-ACCEPTS-UNKNOWN-DECISION	WITNESSED (proved)	the frozen rule commits even pdecUnknown blocks under true
33	OLD-ACCEPTS-CALLER-CONFIG	NOT MECHANIZED	the init-chained F4 witness; nested-keccak #Ceil blocker (§4 (iii)); attempt + log retained; F4 is witnessed single-step by 30, by the audit, and concretely by Python CM4

The headline in full. R1-ACCEPT-IMPLIES-COMMITP's requires-clause is the pAcceptR1 rule's guard copied verbatim (receipt match, derived-decision validity, decision binding, execution validity, policy equality, context equality, parent link, slot link, config witness). The destination is `CommitP(...) ==K true`. The proof is by definitional reduction of `Commit_P` into its four conjuncts, each of which is literally a requires-conjunct — the operational rule provides every premise, and nothing is assumed. This is

the formal answer to finding F3: the theorem now describes the machine.

Non-vacuity. Safety was not obtained by disabling the transition: R1-VALID-ACCEPT and R1-PIPELINE-HONEST prove the honest path fires and commits; the Python mirror runs the same path with real keccak digests (CM6) and chains a second block.

6. Countermodels CM1-CM8

K side (claims 5–11, 14, 16–18 above): the forged-verdict, no-receipt/empty-pool, wrong-candidate, stale-head, wrong-slot and config-substitution rejections with state-unchanged destinations; the two end-to-end pipelines (honest CM6, forged CM1).

krun transcripts (phasesi-r1/transcripts/demos/, Haskell backend; digest-valued cells appear as unevaluated `Keccak256raw(...)` applications — the documented crypto abstraction; the backend may emit symbolic `#Ceil/#Or` disjuncts for undecidable conditions):

- T1-BEFORE-frozen-forged-verdict-ACCEPTS.txt — the frozen model accepts a `pbDecision = 0` block under `SRW3OK = true`; the chain extends (slot 0 committed, counter 1). The vulnerability, live.
- T2-AFTER-repaired-forged-verdict-REJECTS.txt — the repaired model, the same attack with the block even claiming `pbDecision = 1`: the receipt records "invalid-policy"; counter stays 0; no committed block; the receipt remains unconsumed.
- T3-AFTER-no-receipt-REJECTS.txt — no gate evaluation at all: fail-closed.
- T4-AFTER-config-substitution-REJECTS.txt — declared version 8 against pinned version 7: fail-closed.
- T5-AFTER-honest-path-hs-boundary.txt — the honest path under the `hs` backend: the digest equalities are undecidable, so the output is a symbolic disjunction in which the accept branch is conditionally enabled (the transition fires exactly when the real-keccak equalities hold). The branch is decided by the Python mirror (CM6) — the disclosed boundary in action, not a model defect.

Python side (phasesi-r1/python/run_r1_attacks.py, real keccak-256, **48/48 PASS**, transcript `transcripts/attacks/r1-attack-suite.txt`): each case runs on BOTH logics — the frozen logic witnesses the before-side acceptance, the repaired logic rejects with byte-identical state:

Case	Frozen logic (before)	Repaired logic (after)
CM1 forged-true (gate said invalid-policy; block claims decision=1)	accepts	rejects ; state unchanged; receipt remains
CM2 forged-true-context (stale context digest)	accepts	rejects (authorized-projection inequality, real digests)
CM3 forged-true-execution (mismatched execution results)	accepts	rejects (receipt/execution binding)
CM4 config-substitution (init C1, commit under C2)	commits under the caller-chosen C2	rejects twice (CV witness; then the C2-bound policy commitment fails on the real digest even with CV=7)
CM5 forged-receipt / replay	— (no receipts in the frozen model)	different-block receipt does not authorize; replay of a consumed receipt rejects
CM6 honest	—	commits ; head = <code>BlockCommitI(B, CFG1)</code> (real keccak); second block chains
CM7 unknown-evidence / unknown verdict	accepts <code>pdecUnknown</code> blocks	rejects (no receipt / garbage verdict $\Rightarrow$ <code>pdecReject</code>)
CM8 stale-reorg (receipt from displaced head)	accepts stale-evidence replays	rejects (head/slot freshness of the receipt)

A subtle and important CM1 observation (recorded in the suite output): the consistently forged block — every field forged to match the pinned configuration, decision field claimed valid — **satisfies the field predicate** `Commit_P`. That is precisely the frozen model's blind spot: its only security input was the caller Boolean, and the theorem ladder assumed the verdict implied `Commit_P`. The repaired model does not need the block's self-description to be honest: the receipt binds the gate's actual verdict for that exact candidate, and `R1-DECISION-BOUND` ties the field to it. The honestly-rendered rejected block (decision = 0) is additionally excluded by the predicate itself.

7. Python/K correspondence (honest)

- The Python canonical byte encodings (0x9C/0x9D/0x9E/0xA5/0xA6/0x9F tagged preimages, I2B4, keccak-256 with original padding) are compared against the preimages recorded in the K krun transcripts (KX group, 5 checks): the `ProtoRoot` preimage in T2 is byte-identical to Python's `ProtoCfgCanon(CFG1)`; the receipt's bound head in T2 carries the same preimage; the `BlockCommit` preimages in T5 share the Python canonical prefix (0x9E || block fields) and suffix (decision || slot) — the `hs` output is a symbolic disjunction over the undecidable digest equalities, and in the collision-conditional branch the policy-commitment position carries the block's claimed field (0x88) rather than the genuine commitment; the real-keccak branch is decided by Python (CM6). This is stated as observed, not smoothed over.
- The decision mapping, the guard conjunct order, and the state-update shape (receipt consumed, counter +1, head := `BlockCommitI`) are the same in both; the K model stores the pin/pool in the reserved negative-key region where the Python model uses explicit fields — a representation difference documented in §4, with identical semantics (verified by the matching pipelines).
- The Python suite is an executable reference, **not** a proof: every "proved" label in this report refers to a K claim; Python results are DEMONSTRATED.

8. Trust boundaries and classification

Evidence classes used throughout: **PROVED BY K** (`hs` backend, claim id), **WITNESSED** (proved claim over the frozen model demonstrating a defect), **DEMONSTRATED** (Python/krun execution), **NOT MECHANIZED** (attempt + exact blocker retained), **ASSUMED** (named trust boundary).

1. **Gate-verdict provenance (ASSUMED, named).** The verdict string is the abstracted output of the frozen Phase 1G `VerifyLineageG`. The repaired model binds the verdict to the candidate and derives the decision, but the unforgeability of "valid-g" as a token rests on the frozen gate module (mechanized separately in Phase 1G: 21 atomic conjuncts, first-fail, its own accept/reject claims) being the only producer. In the repaired model the only producer of receipts is `pGateEval` — the boundary is the module edge, stated here and in the audit.
2. **Keccak collision resistance (ASSUMED, named).** The policy/context digest equalities are symbolic constraints on the `hs` backend (no `Keccak256raw` evaluators — finding 1C-1); deciding them concretely is demonstrated by the Python mirror and rests on collision resistance. Two configurations identical in all decidable fields but differing in `policyD` are distinguished only by this assumption (CM4's second rejection demonstrates the concrete inequality).
3. **NOT MECHANIZED items (with exact blockers).** `R1-ALL` — kore circularity implication-check abort (same as the frozen `PI7-ALL`; base/step/reject pieces are PROVED; the content is DEMONSTRATED by the exhaustive Python suites). `OLD-ACCEPTS-CALLER-CONFIG` — nested-keccak `#Ceil` blocker (§4 (iii)). The LLVM-tier digest instantiation demo — toolchain not re-provisioned (same disclosure as Phase 1I); covered by the Python keccak mirror.
4. **No consensus-integration claim.** As in Phase 1I: this is a protocol-model result at Level III. Nothing here claims Ethereum protocol integration, consensus-layer enforcement, or Level IV.

9. Zero regression

All frozen suites re-run live from this branch, exact match with the Phase 1I/1H-R1 records:

Suite	Result
phase1h/python/test_lineage_r1.py (LID)	11/11 PASS
phase1h/python/test_security_context_r1.py (SCT)	26/26 PASS
phase1i/python/run_attack_matrix.py	30/30 PASS
phase1i/python/run_determinism_i.py	15/15 PASS
phase1i/python/run_reorg_replay.py	17/17 PASS
phase1i/python/run_forkchoice_compare.py	16/16 PASS
phase1i-r1/python/run_r1_attacks.py (new)	48/48 PASS

Frozen K artifacts are byte-stable (the frozen definition kompiles unmodified and proves the frozen claims; phase1i/ is untouched by construction — `git diff e04fbb6 -- phase1i/ phase1h/ phase1g/` is empty).

10. Completion criteria (handoff checklist)

- [x] The old Phase 1I transition is analyzed and its vulnerability is documented and mechanically witnessed (F2/F4; §3, §6).
- [x] Gate result is derived/bound by the operational semantics, not trusted as a caller-supplied Boolean (receipt design; F2 removed from the command surface).
- [x] Protocol configuration is pinned at initialization and cannot be substituted by accept (immutable pin; no config argument; R1-CONFIG-*).
- [x] Forged-true and config-substitution countermodels reject without state mutation (K claims 5/14/18; Python CM1/CM4; krun T2/T4).
- [x] Honest valid transition still commits (non-vacuity: R1-VALID-ACCEPT, R1-PIPELINE-HONEST, Python CM6).
- [x] R1-ACCEPT-IMPLIES-COMMITP is proved **without assuming its conclusion** (the requires-clause is the rule's guard verbatim; Commit_P appears only in the destination).
- [x] PI7-ALL universal closure retried on the repaired model (R1-ALL); actual outcome retained: NOT MECHANIZED, same kore blocker, log kept.
- [x] Python/K bounded fixtures compared honestly (KX group; §7).
- [x] All earlier Phase 1I and Phase 1H-R1 histories remain unchanged (additive tree; zero-regression table above).
- [x] Report, PDF, provenance, manifest, hashes, and all failed attempts committed.
- [x] No new Phase 1J is started automatically.

11. Verdict and next step

PARTIALLY CLOSED. The repaired operational rule is proved to enforce Commit_P under the pinned configuration (the central binding question is closed — CLOSED's criterion is met); the verdict is PARTIALLY CLOSED because the forall-closure remains open (inherited blocker, retried and logged) and soundness rests on the two named boundaries of §8. The frozen model's forged-verdict acceptance is REFUTED as a safe design (witnessed, not assumed) and repaired.

The prior-art novelty audit named in the handoff is explicitly **out of scope here** and is not started; it is the next task after this report, per the handoff's ordering.